

Introduction

If you've spent any time building automations, you already know the real danger isn't the workflow that crashes — it's the one that keeps running while quietly producing wrong data. That's exactly the trap in today's lesson. We're looking at the Set node in n8n, officially labeled "Edit Fields" in the UI, and specifically at a typo that doesn't throw an error, doesn't fail a run, and doesn't show up in any log. It just silently creates a new field and leaves the one you actually wanted untouched.

This matters for anyone working with automation platforms, and it maps directly onto concepts every backend engineer already knows: schema normalization, data shape consistency, and the difference between a validation error and a silent no-op. If you've ever debugged a null value that "should" have been there, you've lived this exact bug in a different codebase. Understanding how n8n handles field names — and why it doesn't validate them — will save you the hour I lost the first time I hit this.

We'll also cover the mechanics you need before that story makes sense: Manual mode vs JSON mode, how to add and rename fields, how types affect downstream filters, and the "Keep Only Set Fields" toggle that reshapes an entire item.

Problem Overview

Here's the situation in plain terms. You're pulling data from an API, and the response looks like this:

```
{
  "user_name": "grace",
  "userAge": "34",
  "debug_flag": true
}
```

Three problems jump out immediately if you're building anything downstream that depends on consistent field names and types:

1. The name field is called `user_name` instead of `name` — every other part of your workflow expects `name`.
2. There's no `email` field at all — you need to derive or add one.
3. A `debug_flag` snuck in — noise that shouldn't reach production logic.

You need a way to reshape this item into something predictable, without writing custom JavaScript or regex, and without touching the nodes that come after it. That's the exact job of the Set node.

Example

Let's say your API keeps handing you inconsistent field names. Today it's `user_name`, tomorrow it might be `full_name`. Your goal is a clean, predictable output:

Input item:

```
{
  "user_name": "grace",
  "userAge": "34",
  "debug_flag": true
}
```

Desired output item:

```
{
  "name": "grace",
  "age": 34,
  "greeting": "Hey grace, welcome back!"
}
```

Walking through what changed: - `user_name` was renamed to `name`. - `userAge` was renamed to `age` **and** converted from text to a number — without this, a filter like "age greater than 18" would silently fail because you'd be comparing a number to a string. - `debug_flag` was dropped entirely, because "Keep Only Set Fields" was switched on. - `greeting` is new, built from an expression that reads the `name` field on the same item — so it updates automatically no matter what name comes in.

Intuition

Here's the mental model that makes the rest of this click: an item in n8n isn't a fixed record with a schema. It's just JSON, and you're allowed to reshape it at any point in the workflow. The next node doesn't know or care that your data started life as `user_name` with a debug flag attached — it only sees whatever shape you hand it when it arrives.

An experienced engineer treats this the same way they'd treat an adapter layer at a system boundary. When you integrate with a third-party API, you don't let that API's naming conventions leak through your entire codebase — you normalize at the edge, once, and everything downstream stays clean. The Set node is that adapter layer for n8n. It answers a single question at each field: what do I call this, what type is it, and where does its value come from?

Once you see it that way, the two modes (Manual and JSON) are just two different editors for the same idea — rows of key/type/value versus a raw object you write yourself. Neither one is "more powerful"; they're trading off visibility for speed. Manual mode is safer when you're unsure what you're building. JSON mode is faster once you already know the shape you want and don't want to click through five rows to get there.

Brute Force Solution

If you didn't have a Set node, you'd reach for a Code node with custom JavaScript (or Python) that rebuilt the item manually:

```
return items.map(item => {
  return {
    json: {
      name: item.json.user_name,
      age: parseInt(item.json.userAge, 10),
      greeting: `Hey ${item.json.user_name}, welcome back!`
    }
  };
});
```

Advantages: - Full control — anything JavaScript can express, you can do here. - No UI limitations; complex conditional reshaping is possible.

Disadvantages: - Requires writing and maintaining code inside a no-code tool, which defeats part of the point. - Harder to read at a glance for anyone reviewing the workflow later. - No built-in field-level UI, so mistakes like typos are just as silent — arguably worse, since there's no visual list of "these are the fields I defined." - Time complexity: $O(n)$ over the items in the batch, same as the Set node — the cost isn't performance, it's maintainability.

Optimal Solution

The Set node gets you the same $O(n)$ transformation with none of the code-maintenance cost, plus a visible list of exactly which fields you're touching. The workflow looks like this:

Step	Action	Why
1	Add a field named <code>name</code> , value = expression pointing at <code>user_name</code>	Establishes the canonical field name
2	Rename <code>userAge</code> to <code>age</code> , change type to Number	Fixes both the label and the type so downstream filters work
3	Add a field <code>greeting</code> , value = expression referencing <code>name</code>	Demonstrates a computed field built from other fields on the same item

Step	Action	Why
4	Toggle "Keep Only Set Fields" on	Drops <code>debug_flag</code> and any other field you didn't explicitly define

The key design decision is step 4. Left off, your new fields join the original ones — useful when you're only patching a couple of fields on an otherwise fine item. Switched on, only the fields you explicitly defined survive. Use it whenever you want a clean, minimal item instead of "original plus extras."

Python Code

Since n8n's Set node is a no-code UI element, there's no literal Python equivalent to run — but here's the logic it performs internally, written as a standalone function. This is useful both for understanding the node's behavior and for reference if you ever need to replicate this transformation in a Python-based pipeline:

```
from typing import Any

def set_fields(
    item: dict[str, Any],
    field_map: dict[str, tuple[str, type]],
    keep_only_set_fields: bool = False,
) -> dict[str, Any]:
    """Reshape an item the way n8n's Set node does.

    field_map maps a new field name to (source_field_name, target_type).
    """
    result: dict[str, Any] = {}

    for new_name, (source_field, target_type) in field_map.items():
        raw_value = item.get(source_field)
        result[new_name] = target_type(raw_value) if raw_value is not None else None

    if not keep_only_set_fields:
        # merge original fields that weren't explicitly overwritten
        for key, value in item.items():
            result.setdefault(key, value)

    return result

def build_greeting(item: dict[str, Any]) -> str:
    return f"Hey {item.get('name', 'there')}, welcome back!"
```

Code Walkthrough

- `field_map` mirrors the rows you'd fill in Manual mode: new field name → (where the value comes from, what type to cast it to).
- `target_type(raw_value)` performs the type conversion — this is the equivalent of switching a field's type from Text to Number in the UI, and it's where the "age" filter bug gets fixed.
- The `keep_only_set_fields` flag directly mirrors the toggle in the node. When `False`, `setdefault` backfills any original field not already in `result`, so nothing is lost. When `True`, that block is skipped entirely and only explicitly set fields remain.
- `build_greeting` is a standalone function to represent an expression field — a value computed from another field already present on the item, evaluated fresh for every item in the batch.

Dry Run

Starting item:

```
item = {"user_name": "grace", "userAge": "34", "debug_flag": True}
```

Field map:

```
field_map = {  
    "name": ("user_name", str),  
    "age": ("userAge", int),  
}
```

Step-by-step: 1. `name = str(item.get("user_name"))` → "grace" 2. `age = int(item.get("userAge"))` → 34 (this is where text becomes a real number) 3. `keep_only_set_fields=True`, so the merge-back step is skipped — `debug_flag` never makes it into `result`. 4. `greeting` is computed separately: `build_greeting({"name": "grace"})` → "Hey grace, welcome back!"

Final item:

```
{"name": "grace", "age": 34, "greeting": "Hey grace, welcome back!"}
```

Matches the target shape exactly.

Complexity Analysis

- **Time:** $O(n \times k)$, where n is the number of items in the batch and k is the number of fields being set. Each item is processed independently, and each field map entry does constant-time

dictionary lookups and casts.

- **Space:** $O(n \times k)$ for the output batch, since a new dictionary is built per item rather than mutating in place.
- Both are correct because there's no nested iteration over items — every item's transformation is independent, which is also why n8n can process large batches without the cost scaling non-linearly.

Alternative Solutions

You could use a Function/Code node for full custom logic, as shown in the brute-force section, or an "Edit Fields" step combined with an "If" node for conditional reshaping. The Set node wins whenever the transformation is static and declarative — same fields, same types, every time. Reach for a Code node only when the reshaping logic depends on runtime conditions the UI can't express (e.g., "rename this field only if another field equals X").

Edge Cases

- **Empty input item:** Expressions referencing a missing field return `undefined` / `null` rather than throwing — always check what your expression does when the source field doesn't exist.
- **Duplicate field names in Manual mode:** the last row wins; earlier rows with the same name are silently overwritten.
- **Renaming to an existing field name:** this overwrites the existing field's value — there's no warning.
- **Numeric strings with leading zeros or currency symbols:** converting "\$34" to Number will fail or produce `NaN`; strip formatting first.
- **Very large batches:** Set node performance scales linearly, but expressions that call external functions per item can slow batch processing — keep expressions pure.

Common Mistakes

1. **Misspelling a field name while renaming it** (`emial` instead of `email`) — n8n doesn't validate against existing fields, so it silently creates a new field instead of erroring.
2. **Leaving a numeric field as Text** — filters like "greater than 18" fail silently because you're comparing a string to a number.
3. **Forgetting "Keep Only Set Fields" is off by default** — assuming a clean item when the original noisy fields are still attached.

4. **Hardcoding a value instead of using an expression** — the field works once, then breaks the moment the source data changes.
5. **Not checking the field name after typing it** — a stray space or wrong case creates a field that looks identical at a glance but doesn't match downstream references.

Interview Questions

- How would you design a data normalization layer for an API with inconsistent field naming?
- What's the tradeoff between failing loudly (throwing errors) and failing silently (creating unexpected fields) in a data pipeline?
- How would you validate a schema transformation step to catch typos before they reach production?
- Why does type coercion matter for filter/comparison logic downstream in a pipeline?

Similar Problems

- **LeetCode 49: Group Anagrams** — similar idea of reshaping data into a canonical form (sorted string) before comparison, the same normalize-once philosophy.
- **LeetCode 217: Contains Duplicate** — relies on consistent typing/hashing, directly related to why type mismatches break comparisons.
- **LeetCode 1189: Maximum Number of Balloons** — another case where building a frequency/shape map upfront simplifies everything downstream.

Key Takeaways

- The Set node's job is to normalize messy data into a predictable shape, once, at the source — not to patch around it three nodes downstream.
- Manual mode and JSON mode are the same tool with different tradeoffs; don't avoid JSON mode just because it looks more intimidating.
- Type matters as much as naming — a field with the wrong type breaks filters silently.
- "Keep Only Set Fields" is the difference between "patch a few fields" and "rebuild the item from scratch."
- n8n does not validate field names against existing ones. A typo doesn't error — it creates a new field and leaves your real target untouched. Always double-check the name after typing it.

Watch the Video

This lesson is easier to follow watching it happen live — seeing Manual mode and JSON mode side by side, and watching the typo bug happen in real time, makes the mental model stick faster than reading alone. Watch it here: {LINK}

About the Series

This is part of **Fun with Learning Technology**, a course series built around one idea: we don't just click buttons, we figure out why they work. Each lesson builds real automation and engineering skills from the ground up — this one on the Set/Edit Fields node is Lesson 6, and it builds directly on the fundamentals covered earlier in the series.

Call To Action

If this saved you from a debugging session at midnight, the subscribe button on YouTube is right there — it helps more than you'd think. Subscribe to the Substack for the write-up version of every lesson, drop a comment with your `emial` typo story (we've all got one), and share this with anyone on your team still hardcoding field names into their workflows.

Quick Review

Set node: when should you reach for it?

Whenever an item's fields need adding, renaming, retyping, or trimming before the next node — reshaping data, not creating it.

Manual Mode vs JSON Mode in Set node?

Manual Mode = point-and-click field editor for simple changes; JSON Mode = write raw JSON for nested/dynamic structures.

What does 'Keep Only Set Fields' actually do?

Drops every field not explicitly defined in this node — output item contains only what you set, nothing inherited.

Core insight behind the Set node's design?

n8n items are just plain JSON objects flowing between nodes, so 'transforming data' is really just reshaping that JSON.

Gotcha: you rename a field with a typo. What happens?

No error — n8n creates a new field with the typo'd name instead of renaming; downstream nodes silently get undefined data.

Real use case for Set node with expressions?

Normalizing an inconsistent API response: pull values from nested/oddly-named fields via expressions into clean, consistent field names.

n8n Lesson 6: The Set (Edit Fields) node (adding, renaming, and transforming fields on items) — free Python cheat sheet